

Verteilte Systeme

Abgabe 2 - Bericht

Aufgabe 1

In der ersten Aufgabe soll ein Bankhaus als verteilte Anwendung umgesetzt werden. Ich habe Claude Opus 4.6 für die Umsetzung der Aufgaben genutzt und habe schrittweise die Lösung erarbeitet.

Der Kern der Anwendung besteht aus den Klassen *Messages*, *AccountNodes* und *DelayedLinks*. *Messages* beinhaltet das Record für den Transfer von Geld zwischen den Knoten. Es beinhaltet auch das Record *Tick()*, welche von Knoten an sich selber gesendet wird, um den nächsten Transfer auszulösen. *StopTransfers()* und *Terminate()* werden für die Terminierung des Systems genutzt.

Die Klasse *AccountNode* erweitert die durch den Simulator bereits gegebene Klasse *Node*. Diese stellt alle für die Kommunikation notwendigen Funktionen zwischen Knoten bereit. *AccountNode* besitzt eine Liste mit allen anderen Knoten, sodass Multicasts und Überweisungen an jeden Knoten durchgeführt werden können. Die Funktion *doTransfer()* ermöglicht das Verschicken von Geld. Es wird dabei sofort das eigene Konto belastet, während der Empfänger erst nach Erhalt sein Saldo erhöht.

Um die Verzögerung zwischen dem Absenden und Empfangen einer Überweisung zu realisieren, wird die Klasse *DelayedLinks* verwendet. *DelayedLinks* läuft als eigener Thread und simuliert den Weg zwischen zwei Knoten in einem System. Die Klasse erhält das Versenden der Nachricht als Runnable und führt dieses erst nach einer Verzögerung aus, welche zufällig bestimmt wird, um ein Verhalten wie in echten Systemen zu simulieren. Versendete Nachrichten bleiben somit im Kanal.

Über die Config-Datei können verschiedene Parameter für Simulation angepasst werden. Darunter fällt unter anderem die Anzahl an Knoten im System, der Grenzwert für die Terminierung, das initiale Startsaldo, die Regelmäßigkeit der versendeten Nachrichten, sowie die simulierte Verzögerung durch *DelayedLinks*. Durch *BankhausSimulation* und *BankhausRun*, kann die Simulation gestartet werden. Diese lesen die Config-Datei ein und starten die Threads. In der Simulation soll keine FIFO-Zustellung gegeben sein. Deshalb wird die Wahl der nächsten Nachricht im Simulator durch *RandomValues.getUniformDistribution()* realisiert, wodurch immer eine zufällige Nachricht aus der Mailbox gewählt wird. Um die Anwendung zu beenden, schickt der Koordinator als erstes über einen Multicast *StopTransfers* an alle Knoten, wodurch keine neuen Überweisungen mehr entstehen. Nach $4 \cdot \text{maxLatenz} + \text{maxTick}$ ms wird dann ein Multicast mit *Terminate* verschickt, wodurch die Prozesse beendet werden.

Aufgabe 2

In der zweiten Aufgabe soll der in der Vorlesung besprochene Schnappschuss-Algorithmus implementiert werden. Dabei soll der lokale Zustand jedes Prozesses und der Kanalzustand im Schnappschuss festgehalten werden. Ich habe mich für das Koordinator-/Einfärbeverfahren entschieden. Dafür wurde die Klasse *SnapshotCoordinator* erweitert. Der Koordinator bestimmt wann ein Snapshot gemacht wird und leitet die Terminierung des Programms ein, sobald eine bestimmte Anzahl an Snapshots vorhanden sind. Die Anzahl lässt sich in der Config-Datei festlegen. Die Terminierung läuft analog zur vorherigen Beschreibung ab. Bei jedem Schnappschuss und bei der Terminierung muss gelten, dass die Summe der Saldos aller Prozesse und Beträge von Transaktionen die noch nicht abgewickelt worden, gleich dem Startsaldo sein muss.

Um den Schnappschuss-Algorithmus umzusetzen, wird *AccountNode* erweitert. Ein Prozess speichert nun die Anzahl an gesendeten und die Anzahl erhaltener Nachrichten für jeden anderen Prozess in zwei HashMaps *sentCount* und *receivedCount* ab. Jeder Prozess speichert die bereits aufgenommenen Snapshots in der Variablen *int color* ab. Dabei ist ein Knoten weiß, wenn $color < k$ und schwarz, wenn $color \geq k$, wobei k die aktuelle Runde ist, bzw. der k -te Schnappschuss. Der Koordinator kann einen *SnapshotRequest* senden und als Antwort von den Prozessen einen *StateReport* erhalten. Beim Verschicken einer Nachricht wird nun auch die Farbe mitgegeben. Die Nachricht *SnapshotRequest* ist schwarz. Beim Erhalten der Nachricht muss nun geprüft werden, welche Farbe die Nachricht hat, und welche Farbe der eigene Prozess hat.

- Fall weiß -> weiß
Es muss nichts unternommen werden. Beide Knoten sind noch weiß, also geschieht die Transaktion vor dem Schnitt. Der Betrag vom Sender wurde bereits abgezogen und muss nur noch dem Empfänger gutgeschrieben werden.
- Fall schwarz -> schwarz
Der Sender hat nach dem Schnitt die Transaktion durchgeführt. Der Betrag der Transaktion ist im Saldo des Senders gewesen und dem Koordinator geschickt worden. Der Saldo des Empfängers wurde ebenfalls schon gemeldet. Nichts zu unternehmen.
- Fall schwarz -> weiß
Der Sender hat sein Saldo schon gemeldet. Der Empfänger muss sein Saldo melden, bevor er die Nachricht verarbeitet, sonst würde der Betrag doppelt gezählt werden. Außerdem muss er vor der Verarbeitung seine Farbe ändern.

- Fall weiß -> schwarz

Der Sender hat sein Konto belastet, die Nachricht gesendet und dann seinen Zustand gemeldet. Der Empfänger hat seinen Zustand beim Erreichen der Nachricht schon gemeldet. Um das „verlorene“ Geld wieder in den konsistenten Zustand des Schnappschusses zu transferieren, wird diese Basisnachricht an den Schnappschuss nachträglich gemeldet. Es gehört zum Zustand des Kanals.

Das Nachmelden von weißen Nachrichten, wie im letzten Fall beschrieben geschieht mithilfe von *record ChannelReport* in der Klasse *Message*. So werden die Nachrichten, die unterwegs auch im Schnappschuss festgehalten. Da jeder Prozess die gesendeten und empfangenen Nachrichten von zwei Knoten *i* und *j* paarweise in einer *HashMap* speichert, weiß der Koordinator durch *sent_i[j]* - *recv_j[i]* wie viele Nachrichten unterwegs sind. Der Koordinator wartet dann, bis alle Nachrichten und Zustände durch *ChannelReport* gemeldet werden, wodurch der Schnappschuss vollständig ist. Jeder Knoten verwaltet seine Attribute selbst, wodurch die Zustände mit den vorher erwähnten Fällen konsistent sind.

Weil es nicht FIFO ist, weiß der Koordinator nicht welche Nachrichten noch unterwegs sind oder nicht. Das führt dazu, dass die *HashMaps* eingesetzt werden, um die Anzahl an fehlenden Nachrichten zu bestimmen und diese nachträglich hinzuzufügen. Der Koordinator prüft im Anschluss für jeden Kanal, ob die erwartete Summe mit der Summe eingetroffener Nachrichten übereinstimmt. Die Alternative wäre bei jedem Empfang eine Bestätigung mit *ID* zurückzugeben und nach den fehlenden Nachrichten zu suchen. Für die Funktionsweise des Einfärbeverfahrens hat nicht FIFO, keine Auswirkungen, da die Information der Farbe in der Nachricht mitgegeben wird. Der Empfänger sieht dann die Farbe und kann sie mit seiner eigenen vergleichen.

Die Annahmen des Verfahrens sind keine Nachrichtenverluste und keine Abstürze. Bei verlorenen Nachrichten würde der Koordinator ewig auf eine nachträgliche Zustandsmeldung warten und niemals terminieren. Wenn ein Prozess abstürzt bevor er seinen Zustand sendet, führt zum gleichen Ergebnis wie gerade beschrieben.

Aufgabe 3

1)

Ausgabe des Koordinators nach einer Runde:

=== Konsistenter Schnappschuss (Runde/Farbe 1) ===

Lokale Zustände:

P0 balance = 499

P1 balance = 675

P2 balance = 1189

Kanalzustände (unterwegs zum Schnittzeitpunkt):

P0 -> P1 Transfer#50(2) [Summe 2]

P0 -> P2 Transfer#46(220) [Summe 220]

P1 -> P2 Transfer#45(174), Transfer#48(16), Transfer#43(141) [Summe 331]

P2 -> P0 Transfer#47(84) [Summe 84]

Summe Konten : 2363

Summe Kanäle : 637 (6 Ueberweisungen unterwegs)

Gesamt : 3000 (S = 3000) -> INVARIANTE ERFUELLT

Kontrollnachrichten: 12 (= 2n + 6 Nachmeldungen), Dauer 393 ms

In der Ausgabe ist zu sehen, dass die Gesamtsumme der Konten und der Kanäle der Startsumme 3000 entspricht. Messwerte mit unterschiedlichen Seeds sind unter *bankhaus/results* zu finden.

Ausschnitt aus *results/schnappschuesse.csv*:

n	tickMax	Runde	S	Konten	Kanäle	Gesamt	konsistent	unterwegs
12	40 ms	1	12000	6256	5744	12000	True	54
12	40 ms	2	12000	5633	6367	12000	True	55
12	40 ms	3	12000	4434	7566	12000	True	53

In der Tabelle ist zu sehen, dass alle Schnitte des Schnappschuss-Algorithmus konsistent sind.

2)

Ausschnitt aus *results/naiv.csv*:

n	tickMax	Seed	Runde	S	Summe	Gesamt	konsistent	unterwegs
3	40 ms	15838	1	3000	2674	3000	False	1
3	40 ms	15838	2	3000	3143	3000	False	1
3	40 ms	15838	3	3000	3000	3000	True	0

Der naive Algorithmus führt zu Inkonsistenzen. Zu sehen ist, dass Geld verloren geht bei einem und bei einem anderen Schnitt mehr Geld dazu kommt. Beim naiven Algorithmus kann es im Bezug auf den konsistenten Schnitt zu zwei Fehlern kommen.

- 1) Das Sendeereignis wird vor dem Schnitt durchgeführt, das Empfangen aber nicht.

Es ist ein konsistenter Schnitt, da $e_2 \in S \wedge e_1 <_k e_2 \Rightarrow e_1 \in S$ gilt. Jedoch ist der Zustand des Systems falsch, da Geld abgezogen wurde, aber nicht mehr hinzugefügt wird. Somit geht Geld verloren.

- 2) Das Sendeereignis ist nach dem Schnitt und das Empfangen davor.

Das ist ein inkonsistenter Schnitt, da das Senden nicht im Schnappschuss ist, während das kausal darauf aufbauende Ereignis des Empfangens drin ist. So wird mehr Geld hinzugefügt.

Ein konsistenter Schnitt enthält nur globale Zustände, bei denen alle kausalen Abhängigkeiten eingehalten werden, was durch die Formel $e_2 \in S \wedge e_1 <_k e_2 \Rightarrow e_1 \in S$ beschrieben wird. In andere Worte gefasst: Wenn e_2 im Schnappschuss ist und e_1 kausal vor e_2 liegt, dann muss auch e_1 im Schnappschuss enthalten sein. Beim naiven Algorithmus werden nur lokale Zustände

abgefragt und Nachrichten in Kanälen ignoriert. Ein solcher Schnappschuss verletzt die Eigenschaft eines konsistenten Schnitts, da Ursache und Wirkung nicht gemeinsam erfasst werden. In der Tabelle sind die zwei Fehlerarten und die daraus resultierende Inkonsistenz zu sehen.

3)

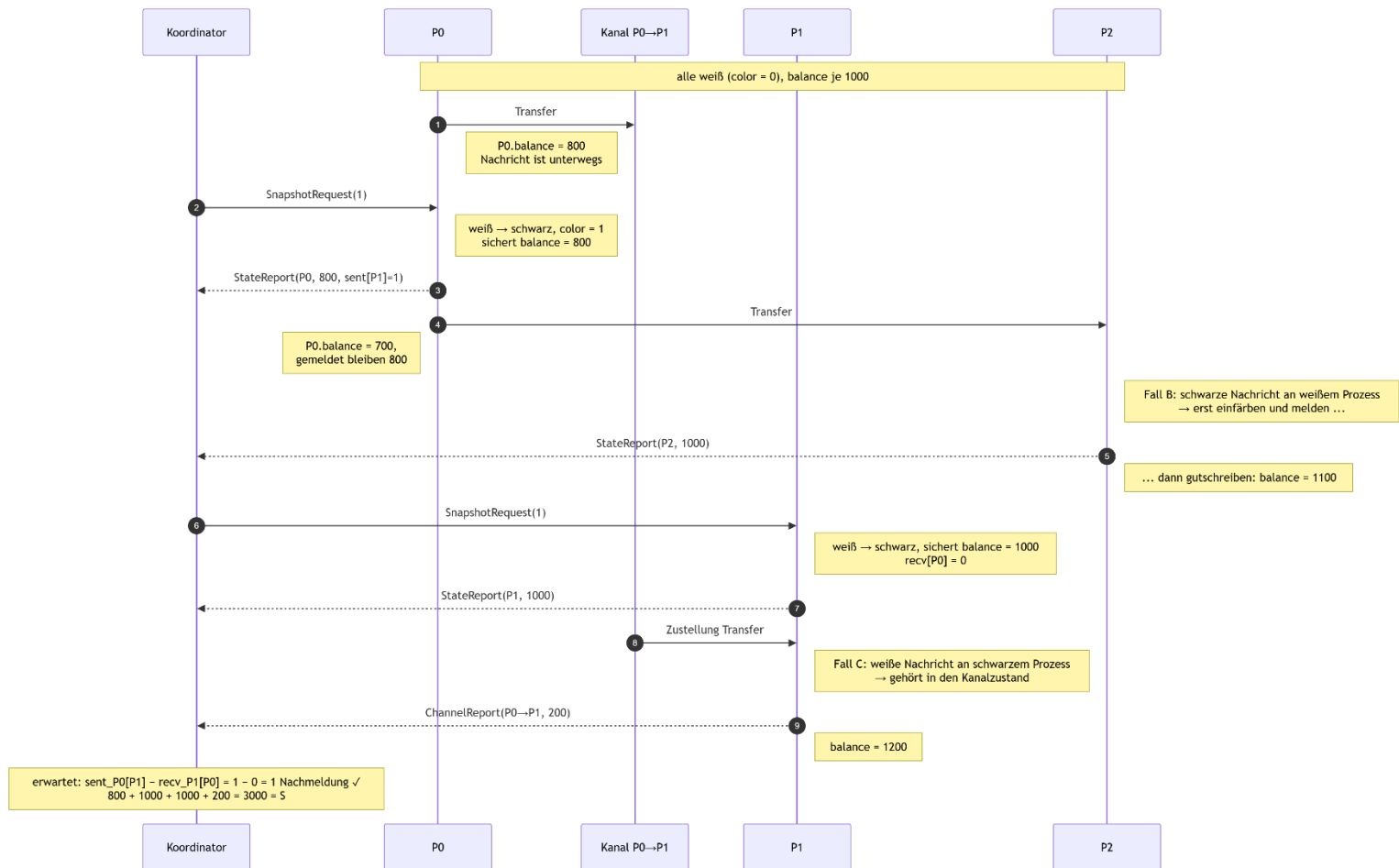
Jede Zeile ist der Mittelwert aus 6 Schnappschüssen (2 Läufe $\times$ 3 Runden):

n	tickMax	unterwegs	Kontrollnachrichten	Dauer des Schnappschusses
3	40 ms	13,7	19,7	412 ms
3	80 ms	6,2	12,2	415 ms
3	120 ms	3,0	9,0	367 ms
5	40 ms	21,8	31,8	453 ms
5	80 ms	10,2	20,2	458 ms
5	120 ms	6,2	16,2	414 ms
8	40 ms	34,8	50,8	464 ms
8	80 ms	17,0	33,0	447 ms
8	120 ms	10,0	26,0	394 ms
12	40 ms	54,0	78,0	492 ms
12	80 ms	25,2	49,2	456 ms
12	120 ms	13,3	37,3	427 ms

Die Kontrollnachrichten pro Schnappschuss entsprechen $2n + m$. Es sind n Kontrollnachrichten, n Antworten und m nachträgliche Nachrichten. Dabei ist m die ausschlaggebende Variable. Durch eine hohe Frequenz steigt die Anzahl der Nachrichten, die unterwegs sind, wodurch mehr nachgeliefert werden müssen. Die Dauer des Schnappschusses bleibt konstant und hängt von der Latenz ab, die eine Nachricht braucht, um anzukommen.

Das hohe Nachrichtenaufkommen des Algorithmus macht ihn zwar nicht langsam, aber führt zu einer hohen Netzwerkauslastung. Ein weiteres potenzielles Bottleneck ist der Koordinator, der das ganze System initiiert und ständig mit diesem kommuniziert. Das System ist zwar verteilt, jedoch muss der Koordinator alle Nachrichten zusammenführen und verarbeiten. Dadurch entsteht ein Single Point of Failure. Fällt der Koordinator aus gibt es keinen Schnappschuss.

Sequenzdiagramm konsistenter Schnitt:



Sequenzdiagramm ohne konsistenten Schnitt:

